Contents

1. Moodle Quiz
2. Häufige Fehler aus Exercise Sheet 2.
3. Beschprechung von Exercise Sheet 3.
4. Implemetation of Maximum Subarray Sum
5. In-Class Aufgaben
6. Theory recap - Seaching and Sorting Algorithmen

Moodle Quizz

Aufgabe 1.

$3n^4 = \Theta(n^5)$

- ○ True
- ○ False

Lösung:

$$\lim_{n\to\infty} \frac{3n^4}{n^5} = \lim_{n\to\infty} \frac{3}{n} = 0 \notin \mathbb{R}^+ \Rightarrow 3n^4 \neq \Theta(n^5) \Rightarrow \text{False}$$

Aufgabe 2.

$2^{1.5n} \leq O(2^n)$

- ○ True
- ○ False

Lösung:

$$\lim_{n\to\infty} \frac{2^{1,5n}}{2^n} = \lim_{n\to\infty} 2^{0,5n} = \infty \Rightarrow 2^{1,5n} \neq O(2^n) \Rightarrow \text{False}$$

Aufgabe 3.

Consider the function $T : \mathbb{N} \to \mathbb{R}^+$ defined by the recurrence

$$T(1) = 10, \quad \text{and} \quad T(n) = T(n-1) + 10 \quad \text{for } n > 1.$$

Enter the smallest value k such that $T(n) \leq O(n^k)$.

Answer:

Lösung:

$T(n) = T(n-1) + 10$

$= T(n-2) + 10 + 10$

$= T(n-3) + 10 + 10 + 10$

$= \dots$

$= T(n-i) + 10i$

$= \dots$

$= T(1) + 10(n-1)$

$= 10 + 10n - 10$

$= 10n$

Wir wissen $T(n) = \Theta(n)$ und $\Theta(n) \subseteq O(n)$

$\Rightarrow$ $k = 1$ ist groß genug

Falls $k < 1$, dann $\lim_{n\to\infty} \frac{T(n)}{n^k} = \lim_{n\to\infty} \frac{10n}{n^k} = \lim_{n\to\infty} 10\, n^{(1-k)} = \infty$,

weil $1 - k > 0$

$\Rightarrow$ $k = 1$

Aufgabe 4.

The goal of a divide-and-conquer algorithm is to…

- a. solve the problem by trying every possible candidate solution.
- b. split the problem into smaller subproblems, solve each, and combine their solutions.
- c. make the problem larger until it becomes easier to solve.

Lösung:

a.) → naive Algorithmus / Brute force

b.) → correct Divide-and-conquer

c.) → generalization

Aufgabe 5

Match the following algorithms for the *Maximum Subarray Sum* problem from the lecture with their running times (where n is the length of the array):

Naive algorithm	Choose...
Divide-and-Conquer algorithm	Choose...
Inductive algorithm	Choose...

Lösung:

Naive algorithm: Berechnet alle mögliche Subarray Summen und wählt die größte davon.

Es gibt $\frac{(n+2)!}{n!\,2!} - n = \frac{(n+2)(n+1)}{2} - n$ solche Summen

⇒ Laufzeit $O(n^2)$

Divide-and-Conquer algorithm: Wir teilen das Array auf linke und rechte teil bzw. - ArrL und ArrR.

Wir lösen das Max-Subarray-Sum Problem für ArrL und ArrR und wir Berechnen die größte Summe die Elemente aus beide ArrL und ArrR enthält.

Aus diese 3 Möglichkeiten, wählen wir die größte



Summe, die aufeinanderfolgende Elemente aus beiden Arrays enthält

Laufzeit: $O(n \log n)$

Inductive algorithm: Für jede Subarray $Arr_i = [a_1, \ldots, a_i]$ mit $i \leq n$, definieren wir:

rand_max[i] = max(0, größte Subarry Summe aus Arr_i, die unbedingt a_i enthält.)

max[i] = die größte Subarray Summe aus Arr_i

Algorithmus: arr[1...n]

```
int rand_max[1...n] ← fill with 0
int max[1...n] ← fill with 0
 rand_max[1] = max(0, arr[1])
  max[1] = max(0, arr[1])
  for i = 2 ... n do
      rand_max[i] = max(0, rand_max[i-1] + arr[i])
      max[i] = max(rand_max[i], max[i-1])
  return max[n]
```

Laufzeit: $O(n)$

Aufgabe 6

The *Maximum Subarray Sum* problem can be solved in time $O(\sqrt{n})$ (where n is the length of the array).

- ○ True
- ○ False

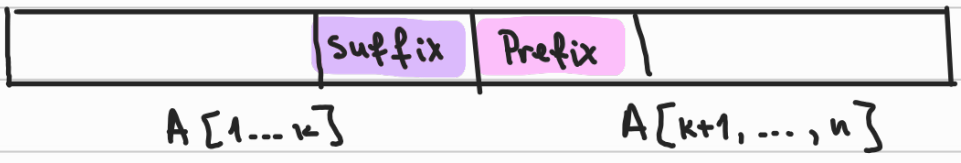
Lösung: Jede Element des Arrays kann das Antwort verändern. Das bedeutet, dass wir müssen alle n Elemente berücksichtigen.
⇒ Jede Algorithmus, der das Maximum Subarray Sum Problem löst hat Laufzeit in $\Omega(n)$
⇒ False

Aufgabe 7.

Let $A[1..n]$ be an input array for the *Maximum Subarray Sum* problem, where n is even. Let $k := n/2$. The maximum sum of a subarray of A that contains at least one element of $A[1..k]$ and at least one element of $A[k+1..n]$ can be computed by...

- ○ a. the maximum non-empty prefix sum of $A[1..k]$ plus the maximum non-empty suffix sum of $A[k+1..n]$.
- ○ b. the maximum non-empty suffix sum of $A[1..k]$ plus the maximum non-empty prefix sum of $A[k+1..n]$.
- ○ c. the maximum non-empty subarray sum of $A[1..k]$ plus the maximum non-empty subarray sum of $A[k+1..n]$.
- ○ d. the sum of all elements of A.

Lösung:



=> b)

Aufgabe 8.

Consider the *search* problem with a given input array $A[1..n]$ and target element b.

If A is *not* sorted, then the linear search algorithm with running time $\Theta(n)$ is optimal.

- ○ True
- ○ False

Lösung: Falls A[1...n] nicht sortiert ist. Falls b kein Element von A ist, dann müssen wir durch alle Elemente von A gehen um sicher zu sein, dass A b nicht enthält.
=> True

Aufgabe 9.

Consider the *search* problem with a given input array $A[1..n]$ and target element b.

For binary search, it does not matter whether A is sorted or not.

- ○ True
- ○ False

Lösung: Binary search is nur möglich, wenn das Array sortiert ist

=> False

Aufgabe 10

We can check whether or not an array of length n is already sorted in time $\Theta(n)$.

- ○ True
- ○ False

Lösung: Sei A[1... n] ein beliebiges Array aus Zahlen.
Wir können für alle $i \in \{2, \dots, n\}$ testen ob $a_{i-1} \leq a_i$ ($a_{i-1} \geq a_i$)
Falls eine von diesen zwei Ungleichungen gilt, dann ist das Array sortiert.

```
Algorithm: Sorted A[1...n]
  Bool sorted ← true
  for i in 2... n do
     if A[i-1] ≥ A[i] then
          sorted ← false
          Break
  if sorted then
      return true
  sorted ← true
  for i in 2 ... n do
```

```
if A[i-1] ≤ A[i] then
    sorted ← false
    break
return sorted
```

Laufzeit: $O(n)$ ⇒ True

Aufgabe 11

The Bubble-Sort algorithm sorts the elements of a given array by...

- ○ a. always comparing the first and the last element.
- ○ b. always comparing the smallest and the largest element.
- ○ c. always comparing neighboring elements.

Lösung: c): Wir sehen das in 5. Theory recap

Aufgabe 12

Algorithm 1

```
i ← 1
while i ≤ 3n do
    f()
    i ← i + 1
j ← 0
while j ≤ n² do
    f()
    f()
    f()
    j ← j + 1
```

Which of the following expressions correctly describes the exact number of calls to f?

- ○ a. $\sum_{i=1}^{3n} 1 + \sum_{j=1}^{n^2} 3$
- ○ b. $\sum_{i=1}^{3n} 1 + \sum_{j=0}^{n^2} 3$
- ○ c. $\sum_{i=1}^{3n} \sum_{j=1}^{n^2} 4$

Lösung:

Erste while: $\sum_{i=1}^{3n} 1 + \sum_{j=0}^{n^2} 3 \Rightarrow$ B.1

Häufige Fehler aus Exercise Sheet 2.

Exercise 2.2.

Algorithm 1 Recursive maximum

if $n = 1$ **then**
  return a_1
else ▷ $n \geq 2$
  Apply this algorithm recursively and find the maximum element a' among $a_1, \ldots, a_{n-1}$
  if $a' > a_n$ **then**
    return a'
  else ▷ $a' \leq a_n$
    return a_n

Wenn wir Induktion über ein Algorithmus machen, ist es wichtig für jede Teil von unsere Beweis zu sagen:

i) Welche teil des Algorithmus betrachten wir, wenn wir uns in diesem Teil der Lösung Befinden

ii) Warum ist es richtig diesen Teil des Algorithmus zu Betrachten.

Für Aufgabe 2.2. habt ihr alle richtig erkennt, dass wir uns für $n=1$ im Basisfall in der if-case Befinden und für den Schrittfall (Induktionsschritt) in der else-case, aber ihr müsst auch erklären warum das gilt.

Exercise 2.3

Exercise 2.3 *O-notation quiz* **(1 point)**.

(a) For all the following functions the variable n ranges over $\mathbb{N}$. Prove or disprove the following statements. Justify your answer using Theorems 1 and/or 2.

(1) $2n^5 + 10n^2 \leq O(\frac{1}{100}n^6)$

(2) $n^{10} + 2n^2 + 7 \leq O(100n^9)$

(3) $e^{1.2n} \leq O(e^n)$

→ (4)* $n^{\frac{2n+3}{n+1}} \leq O(n^2)$

Wrong solution (a)(4):

$$\lim_{n\to\infty} \frac{n^{\frac{2n+3}{n+1}}}{n^2} = \lim_{n\to\infty} n^{\frac{2n+3}{n+1}-2} = \lim_{n\to\infty} n^{2-\frac{1}{n+1}-2} = \lim_{n\to\infty} n^{\frac{1}{n+1}} =$$

$$n^{\lim_{n\to\infty}\frac{1}{n+1}} = n^0 = 1$$

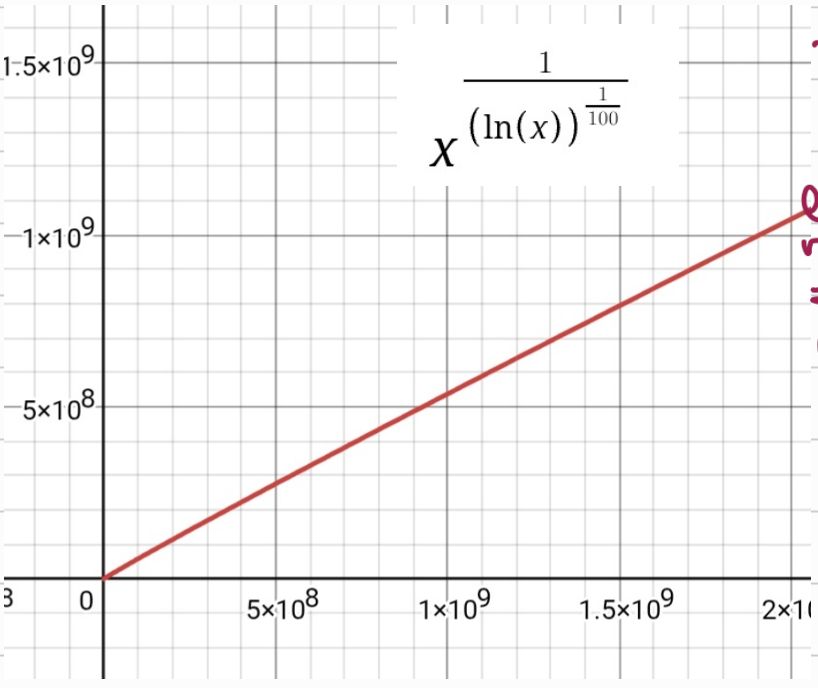
The idea that when we have $\lim_{n\to\infty} n^{\frac{1}{n+1}}$ is the same as $\lim_{n\to\infty} n^0$, because $\lim_{n\to\infty}\frac{1}{n+1}=0$ is wrong.

Let us consider the function $r(n)=s(n)^{t(n)}$, where $s(n)$ and $t(n)$ are bot functions.
It is incorrect to say
$\lim_{n\to\infty} s(n)^{t(n)} = s(n)^{\lim_{n\to\infty} t(n)}$.

Let $r(n) = n^{\frac{1}{\sqrt[100]{e^n n}}}$, where $s(n)=n$, $t(n)=\frac{1}{\sqrt[100]{e^n n}}$.
The functions $s(n)$ and $t(n)$ have different "effect" on the limit $\lim_{n\to\infty} s(n)^{t(n)} = n^{\frac{1}{\sqrt[100]{e^n n}}}$.

By different effect we mean that when $n\to\infty$
$s(n)\to\infty$ and it would "push" $\lim_{n\to\infty} s(n)^{t(n)}$ to go to ∞
$t(n)\to 0$ and it would "push" $\lim_{n\to\infty} s(n)^{t(n)}$ to go to 1.

The behaviour of the function $r(n)=s(n)^{t(n)}$ when n goes to ∞ depends on whether $s(n)$ approaches ∞ faster or $t(n)$ approaches 0 faster.
We cosider the graph of the function.



From the graph we can see that $\lim_{n\to\infty} n^{\frac{1}{\sqrt[100]{\ln n}}} = \infty$, despite the fact that $\lim_{n\to\infty} \frac{1}{\sqrt[100]{\ln n}} = 0$

3. Besprechung von Exercise Sheet 3

Exercise 3.4

Exercise 3.4 *Iterative squaring.*

In this exercise you are going to develop an algorithm to compute powers a^n, with $a \in \mathbb{Z}$ and $n \in \mathbb{N}$, efficiently.

(a) Assume that n is even, and that you already know an algorithm $A_{n/2}(a)$ that efficiently computes $a^{n/2}$, i.e., $A_{n/2}(a) = a^{n/2}$.

Given the algorithm $A_{n/2}$, design an efficient algorithm $A_n(a)$ that computes a^n. (You don't need to argue correctness or runtime.)

(b) Let $n = 2^k$, for $k \in \mathbb{N}_0$. Find an algorithm that computes a^n efficiently. Describe your algorithm using pseudo-code. (You don't need to argue correctness or runtime.)

(c) Determine the number of integer multiplications required by your algorithm for part (b) in O-notation. You may assume that bookkeeping operations don't cost anything. This includes handling of counters, computing $n/2$ from n, etc.

(d) Let Power(a, n) denote your algorithm for the computation of a^n from part b). Prove the correctness of your algorithm via mathematical induction for all $n \in \mathbb{N}$ that are powers of two.

In other words: show that Power$(a, n) = a^n$ for all $n \in \mathbb{N}$ of the form $n = 2^k$ for some $k \in \mathbb{N}_0$.

In your solution, you should address the base case, the induction hypothesis and the induction step.

(e)* Design an algorithm that can compute a^n for a general $n \in \mathbb{N}$, i.e., n does not need to be a power of two. The algorithm should use $O(\log n)$ integer multiplications for $n \geq 2$. You don't need to argue about correctness or runtime.

Hint: *Generalize the idea from part (a) to the case where n is odd, i.e., there exists $k \in \mathbb{N}_0$ such that $n = 2k + 1$.*

Exercise 3.4.

a) Student A und Student B haben 2 unterschiedliche Lösungen geschrieben?

Student A $A_n(a)$:

```
x ← A_{n/2}(a)
return x * x
```

Student B $A_n(a)$:

```
return A_{n/2}(a) * A_{n/2}(a)
```

Machen beide Algorithmen dasselbe und haben sie dieselbe Laufzeit?

b) Student A und Student B haben die Folgende zwei Algorithmen etwikelt:

Student A Power(a, n)

```
if n = 1
    return a
x ← Pow(a, n/2)
return x * x
```

Student B Power(a, n)

```
if n = 1
    return a
return Power(a, n/2) * Power(a, n/2)
```

c) Laufzeitanalyse

Sei $T_A(n)$ die Anzahl von Multiplikationen, die wir brauchen für den Algorithmus Student A Power(a, n)

$T_A(1) = 0$

$\Rightarrow T_A(n) = T_A(\frac{n}{2}) + 1$



$$= T_A\left(\frac{n}{4}\right) + 1 + 1$$
$$= \overbrace{T_A\left(\frac{n}{8}\right) + 1}^{} + 1 + 1$$

$$= \dots$$
$$= T_A\left(\frac{n}{2^r}\right) + r \cdot 1$$

$$= \dots$$
$$= T_A\left(\frac{n}{2^k}\right) + k \cdot 1, \text{ wobei } n = 2^k \text{ und } \frac{n}{2^k} = 1$$
$$= T_A(1) + \log n \quad (n = 2^k \Rightarrow \log n = \log 2^k = k)$$
$$= 0 + \log n$$
$$= \log n$$
$$\Rightarrow T_A(n) \leq O(\log n)$$

Sei $T_B(n)$ die Anzahl von Multiplikationen, die wir Brauchen für den Algorithmus StudentB Power(a, n)

$T_B(1) = 0$

$$\Rightarrow T_B(n) = 2 * T_B\left(\frac{n}{2}\right) + 1$$
$$= 2 * \left(2 * T_B\left(\frac{n}{4}\right) + 1\right) + 1$$
$$= 2 * \left(2 * \left(2 * T_B\left(\frac{n}{8}\right) + 1\right) + 1\right) + 1$$
$$= \dots$$
$$= 2^r \cdot T_B\left(\frac{n}{2^r}\right) + 2^{r-1} + 4 + 2 + 1$$
$$= \dots$$
$$= 2^k \cdot T_B\left(\frac{n}{2^k}\right) + 2^{k-1} + 2^{k-2} + \dots + 4 + 2 + 1, \text{ wobei } n = 2^k$$
$$= 2^k \cdot T_B(1) + \sum_{i=0}^{k-1} 2^i \qquad \Rightarrow \frac{n}{2^k} = 1 \text{ und}$$
$$= 2^k \cdot 0 + \sum_{i=1}^{k+1} 2^i \qquad k = \log n$$
$$= \sum_{i=1}^{k-1} 2^i$$
$$= 2^k - 1$$
$$= n - 1$$
$$\Rightarrow T_B(n) \leq O(n)$$

4. Implementation of Maximum Subarray Sum

Aus der Vorlesung.

Algorithm: Max_Sub-Sum(Arr[1,...,n], n]

```
int rand_max[1,...,n] ← fill with 0
int max[1,...,n] ← fill with 0
rand_max[1] = max(0, Arr[1])
max[1] = max(0, Arr[1])
for i = 2...n do
    rand_max[i] = max(rand_max[i-1] + Arr[i], 0)
    max[i] = max(max[i-1], rand_max[i])
return max[n]
```

Algorithm: Max_Sub_Sum(Arr[1,...,n], n)

```
int rand_max[1,...,n] ← fill with 0
int max[1,...,n] ← fill with 0
rand_max[1] = Arr[1]
max[1] = max(0, Arr[1])
for i = 2...n do
    rand_max[i] = max(rand_max[i-1] + Arr[i], Arr[i])
    max[i] = max(max[i-1], rand_max[i])
return max[n]
```

Beide sind Korrekt.

5. In-Class Exercises

Übung 1.

Sei $S_n = \sum_{i=1}^{n} i^k$. Berechnen Sie $r \in \mathbb{N}$ so dass $S_n = \Theta(n^r)$ für alle $n \in \mathbb{N}$

Lösung:

Übung 2:

Sei $f(n) = n^2 + (-1)^n n^2 + n$ und $g(n) = n$.

Gilt dass $f(n) \leq O(g(n))$?

Lösung:

Sei $n = 2k$, $k \in \mathbb{N}$, dann

$f(n) = n^2 + 1 \cdot n^2 + n = 2n^2 + n$

Wir wollen eine konstante C finden so dass

$\forall n \in \mathbb{N}$: $f(n) \leq C \cdot g(n)$

$f(n) \leq C \cdot g(n)$

$\Leftrightarrow 2n^2 + n \leq C \cdot n \quad | : n \ (n > 0)$

$\Leftrightarrow 2n + 1 \leq C$

Das bedeutet, dass C unendlich groß werden muss, aber C ist nur eine Konstante und das ist unmöglich

5. Theory Recap - Sortind an Searching Algorithms

5.1. Suchen bei Unsortierte Arrays

LINEAR-SEARCH$(A[1..n], b)$ — LINEARE SUCHE

```
1 for i ← 1, 2, ..., n do
2     if A[i] = b then return i
3 return "nicht gefunden"
```

Laufzeit: $\Theta(n)$

5.2. Suchen in Sortierte Arrays: Sinäre Suche

BINARY-SEARCH$(A[1..n], b)$

```
1 ℓ ← 1; r ← n                          ▷ Initialer Suchbereich
2 while ℓ ≤ r do
3     m ← ⌊(ℓ + r)/2⌋
4     if A[m] = b then return m          ▷ Element gefunden
5     else if A[m] > b then r ← m − 1    ▷ Suche links weiter
6     else ℓ ← m + 1                     ▷ Suche rechts weiter
7 return "Nicht vorhanden"
```

Laufzeit: $O(\log n)$

5.3. Sortieren

SORTED(A) — PRÜFE SORTIERTHEIT

```
1 for i ← 1, 2, ..., n − 1 do
2     if A[i] > A[i + 1] then return false
3 return true
```

Laufzeit: $\Theta(n)$

5.3.1. Bubble - Sort

BUBBLE-SORT($A[1..n]$) — BUBBLE-SORT

```
1 for j ← 1, 2, ..., n do
2     for i ← 1, 2, ..., n − 1 do
3         if A[i] > A[i + 1] then
4             tausche A[i] und A[i + 1]
```

Laufzeit Die Laufzeit von Bubble-Sort ist:

- $\Theta(n^2)$ Vergleiche,
- $O(n^2)$ Vertauschungen.

Invariant: Nach j Durchläufen der äusseren Schleife sind die j grössten Elemente am richtigen Ort.

5.3.2. Selection -Sort

SELECTION-SORT($A[1..n]$) — SELECTION-SORT

```
1 for j ← n, n − 1, ..., 1 do
2     k ← Index des Maximums in A[1, ..., j]
3     tausche A[k] und A[j]
```

Laufzeit:

- $\Theta(n^2)$ Vergleiche,
- $O(n)$ Vertauschungen.

Invariant: Nach j Durchläufen der äusseren Schleife sind die j grössten Elemente am richtigen Ort.

5.3.3. Insertion - Sort

INSERTION-SORT

INSERTION-SORT($A[1..n]$)

```
1 for j ← 2, 3, ..., n do
2     k ← kleinster Index in {1, ..., j − 1} mit A[j] ≤ A[k]
                                        ▷ A[j] gehört an diese Stelle k
3     x ← A[j]                          ▷ merke A[j]
4     verschiebe A[k, ..., j − 1] nach A[k + 1, ..., j]
5     A[k] ← x
```

Laufzeit:

- $O(n \log n)$ Vergleiche,
- $O(n^2)$ Vertauschungen.

Invariant

Nach j Durchläufen der äusseren Schleife ist das Teilarray $A[1, \ldots, j]$ sortiert (es enthält aber nicht zwangsläufig die j kleinsten Elemente des Arrays).